TCS Interview Questions

EXP: 4 Years
CTC: 15 LPA
Profile: Data Engineer

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

PYSPARK PULSE

What is the Difference Between RDD, DataFrame, and Dataset?

Feature	RDD (Resilient Distributed Dataset)	DataFrame	Dataset
Abstraction Level	Low-level (functional programming)	High-level (tabular, like Pandas)	Combination of both
API Support	Java, Scala, Python	Java, Scala, Python	Java, Scala
Performance	Slow due to Java serialization	Optimized with Catalyst optimizer	Optimized with type-safety
Schema	No schema	Has schema with named columns	Has schema with compile-time type safety
Optimization	No optimization, manual tuning needed	Catalyst optimizer & Tungsten execution	Catalyst optimizer & Tungsten execution
Type Safety	No	No	Yes (only in Scala and Java)
Use Case	Fine-grained control over computation	SQL-like operations	Strongly typed data processing

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

How Does PySpark Lazy Evaluation Work?

Transformations are not executed immediately.
Actions trigger execution.
Optimizes query plans before execution.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

What are Transformations vs. Actions in PySpark?

Transformations: Lazy, return new RDDs/DataFrames (e.g., map, filter, select).
Actions: Trigger execution (e.g., show, count, collect).

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

Why Do We Use SparkContext & SparkSession?

SparkContext: Used for RDDs in older versions.
SparkSession: Unified API (RDD, DataFrame, SQL), recommended from Spark 2.0 onwards.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

How Does PySpark Handle Schema Inference?

Automatic schema inference for structured data (CSV, JSON, Parquet).
Explicit schema definition using StructType for consistency.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

What Are Broadcast Variables and When Should You Use Them?

Broadcast variables are cached on worker nodes.
Useful for joins with small lookup tables to reduce network traffic.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

How Do You Optimize Joins in PySpark?

Broadcast join for small tables.
Sort-Merge join for large datasets.
Shuffle Hash join when one dataset fits in memory.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

What Are Shuffle Operations and How to Minimize Them?

Shuffle operations occur during:

- Wide transformations (e.g., groupBy(), join(), reduceByKey()).
- Data movement across nodes.

How to minimize shuffle?

- Use coalesce() instead of repartition() to reduce partitions efficiently.
- Use broadcast() for small datasets.
- Avoid unnecessary groupBy() and distinct().
- Use map-side joins instead of shuffle joins.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

What is Partitioning and How Does It Impact Performance?

Partitioning distributes data across nodes.
Too few partitions → data skew.
Too many partitions → high overhead.
Use coalesce() to optimize partitions.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

When Should You Use cache() vs persist() and Why?

cache(): Stores data in memory only.
persist(): Can store data in memory, disk, or both.
Use persist() when data is too large for memory.

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse

When Should You Use cache() vs persist() and Why?

Feature	cache()	persist()
Storage Level	Stores data in memory	Can store in memory, disk, or both
Persistence Level	MEMORY_ONLY	MEMORY_AND_DISK, DISK_ONLY, etc.
Recalculation	Avoids recomputation	Avoids recomputation, with flexibility
Best Use Case	When data fits in memory	When data is too large for memory
df.cache() # Stores in memory only		
df.persist(storageLevel="MEMORY_AND_DISK") # Stores in memory and spills to disk if needed		

Linkedin: [Priyam Jain | LinkedIn]
YouTube: [pysparkpulse]

Difference Between EMR and AWS Glue

Feature	EMR (Elastic MapReduce)	AWS Glue
Purpose	Big data processing using Spark, Hadoop, Presto, Hive, etc.	Serverless ETL for data preparation and transformation.
Infrastructure	Requires cluster setup (EC2 instances, auto-scaling available).	Fully managed, serverless (no infrastructure management).
Use Case	Batch processing, ML workloads, interactive analytics.	ETL jobs, data cataloging, schema discovery.
Scaling	Manual or auto-scaling of EC2 instances.	Auto-scales dynamically (fully serverless).
Performance	Faster for large, complex processing due to dedicated resources.	Optimized for ETL but may be slower for heavy computation.
Cost Model	Pay for EC2 instances and other AWS resources used.	Pay-per-use pricing (charged per Data Processing Unit - DPU).
Flexibility	Highly customizable with full control over Spark/Hadoop settings.	Limited customization but easy to use for ETL tasks.

Linkedin: [Priyam Jain | LinkedIn]
YouTube: [pysparkpulse]

PYSPARK PULSE

Use of AWS Lambda in My Project

Give answer accordingly

Linkedin: Priyam Jain | LinkedIn
YouTube: pysparkpulse